

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра Програмної інженерії

Звіт

з лабораторної роботи №4

з дисципліни: «Поглиблене вивчення JavaScript»

з теми: «Розробка застосунку реєстрації на події (розширені можливості Vue)»

Виконав:

ст. гр. ПЗПІ-23-2

Малишкін Андрій

Перевірів:

посада викладача (уточнити)

ПІБ викладача (уточнити)

4 РОЗРОБКА ЗАСТОСУНКУ РЕЄСТРАЦІЇ НА ПОДІЇ (РОЗШИРЕНІ МОЖЛИВОСТІ VUE)

4.1 Мета роботи

Метою роботи є закріплення та поглиблення знань з розробки застосунків на Vue 3, засвоєння принципів використання Composition API, організації логіки за допомогою composables, реалізація багатосторінкових застосунків, а також формування навичок роботи з асинхронними операціями та збереженням даних.

4.2 Завдання

У межах лабораторної роботи необхідно:

- реалізувати багатосторінковий застосунок реєстрації на події;
- створити список подій та сторінку деталей;
- реалізувати форму реєстрації з валідацією;
- використати composables для роботи з подіями, реєстраціями та localStorage;
- реалізувати асинхронну імітацію запиту до сервера;
- забезпечити оптимістичне оновлення інтерфейсу та відкат змін при помилці;
- відобразити список зареєстрованих учасників;
- застосувати Teleport та KeepAlive.

4.3 Хід роботи

4.3.1 Теоретичні відомості

Було розглянуто Composition API як підхід до групування пов'язаної логіки у Vue 3. Composables дозволяють інкапсулювати роботу з даними та повторно використовувати її у різних компонентах.

Для багатосторінкової навігації застосовано Vue Router з маршрутами для списку подій, деталей та реєстрації. Для збереження даних між сесіями використано «localStorage» через окремий composable.

Окремо було опрацьовано оптимістичний інтерфейс: результат дії відображається одразу, а при помилці асинхронного запиту виконується відкат

до попереднього стану. Додатково застосовано Teleport для toast-повідомлень та KeepAlive для кешування списку подій.

4.3.2 Етапи реалізації застосунку

Під час практичної реалізації було послідовно виконано такі етапи:

- підготовлено проєкт Vue 3 + TypeScript у директорії «src»;
- налаштовано Vue Router з маршрутами «/», «/events/:id», «/events/:id/register»;
- створено composables «useLocalStorage», «useEvents», «useRegistration»;
- реалізовано mock API «registerForEvent» з затримкою та імітованими помилками;
- реалізовано сторінки списку, деталей та реєстрації;
- додано валідацію форми та перевірку дублікатів email;
- реалізовано оптимістичне додавання реєстрації з відкатом при невдачі;
- додано Teleport-toasts та KeepAlive для «EventListView»;
- оформлено інтерфейс у темній темі з Tailwind CSS та shadcn-подібними UI-компонентами.

4.3.3 Опис архітектури та компонентів

Застосовано модульну структуру:

- «composables/useEvents.ts»: каталог подій та допоміжні методи;
- «composables/useRegistration.ts»: логіка реєстрації, валідація, optimistic UI;
- «composables/useLocalStorage.ts»: серіалізація даних у браузері;
- «composables/registrationState.ts»: спільний реактивний стан реєстрацій;
- «api/registration.ts»: імітація асинхронного серверного запиту;
- «views/EventListView.vue», «EventDetailView.vue», «EventRegisterView.vue»: сторінки застосунку;
- «components/ToastHost.vue»: Teleport-повідомлення;
- «UiButton.vue», «UiInput.vue», «UiCard.vue»: базові UI-примітиви.

Такий поділ забезпечив повторне використання логіки та розділення відповідальностей між шарами даних і представлення.

4.3.4 Короткі фрагменти програмного коду

Нижче наведено ключові фрагменти реалізації.

4.3.4.1 Оптимістична реєстрація з відкатом

```
const snapshot = [...registrations.value]
registrations.value =
[optimisticRegistration, ...registrations.value]
persistRegistrations()

try {
  const result = await registerForEvent({ eventId, name, email })
  // підтвердження успіху
} catch (error) {
  registrations.value = snapshot
  persistRegistrations()
  pushToast({ type: 'error', message: '...' })
}
```

4.3.4.2 Composable для localStorage

```
export function loadFromStorage<T>({
  key: string,
  isValid: (value: unknown) => value is T,
  fallback: T,
}): T {
  const raw = localStorage.getItem(key)
  const parsed = JSON.parse(raw)
  return isValid(parsed) ? parsed : fallback
}
```

4.4 Результати роботи

У результаті виконання лабораторної роботи отримано працездатний застосунок «Афіша подій» на Vue 3. Було реалізовано перегляд подій, детальну інформацію, реєстрацію з валідацією, збереження у localStorage, асинхронну обробку з оптимістичним UI та список зареєстрованих учасників.

Під час перевірки працездатності було підтверджено:

- коректну роботу всіх маршрутів;
- збереження реєстрацій після перезавантаження;
- відкат змін при імітованій помилці сервера;
- відсутність помилок lint/typescheck у проєкті.

4.5 Висновки

Під час виконання лабораторної роботи було закріплено практичні навички роботи з Composition API, composables, Vue Router, асинхронними операціями та оптимістичним оновленням інтерфейсу. Було отримано досвід побудови масштабованого SPA з розділенням логіки та використанням розширених можливостей Vue.

4.6 Контрольні запитання

а) Що таке Composition API?

Composition API — підхід Vue 3 до організації логіки компонента через функції («ref», «computed», «watch») та composables замість опцій «data», «methods», «computed».

б) Що таке composables?

Composables — функції, що інкапсулюють реактивну логіку та можуть повторно використовуватися у різних компонентах (наприклад, «useRegistration»).

в) Призначення Vue Router.

Vue Router забезпечує маршрутизацію в SPA: відображає потрібний компонент за URL без перезавантаження сторінки.

г) Як реалізується робота з localStorage?

Дані серіалізуються через «JSON.stringify» і зберігаються ключем «localStorage.setItem»; при ініціалізації зчитуються через «getItem» та «JSON.parse» з валідацією.

д) Що таке оптимістичний інтерфейс?

Це підхід, коли UI оновлюється одразу після дії користувача, ще до завершення асинхронного запиту; при помилці зміни відкочуються.

е) Як обробляються помилки в асинхронних операціях?

Асинхронний код обгортається в «try/catch»; у «catch» виконується відкат стану та показ повідомлення користувачу; у «finally» скидається індикатор завантаження.